

OSINT-КОНВЕЙЕР ОБОГАЩЕНИЯ ИНДИКАТОРОВ КОМПРОМЕТАЦИИ НА F#: ОПИСАНИЕ АРХИТЕКТУРЫ И ОБОСНОВАНИЕ ПРОЕКТНЫХ РЕШЕНИЙ

Назначение системы. Рассматриваемая система представляет собой конвейер обработки индикаторов компрометации (*Indicator of Compromise, IOC*), предназначенный для агрегации, нормализации, дедупликации, обогащения и фильтрации разнородных данных, поступающих из гетерогенных источников (фидов) в форматах CSV, JSON и STIX-lite. Конечным артефактом работы конвейера является структурированный отчёт, содержащий для каждого индикатора его каноническое представление, набор источников, метки, результаты обогащения (геолокация, автономная система, обратный DNS, WHOIS) и признак принадлежности к классу ложноположительных срабатываний с указанием причины отнесения.

Задача относится к классу ETL-процессов (*Extract – Transform – Load*) с элементами обогащения данными (*data enrichment*) и применения продукционных правил. Характерными особенностями предметной области являются:

- гетерогенность входных данных – разные фиды используют разные схемы, форматы и семантику;
- высокая степень дублирования – одни и те же индикаторы встречаются в нескольких фидах, часто с частично пересекающимися метаданными;
- необходимость канонизации – дефангированные (*defanged*) представления (1.2.3[.]4, hxxp://...) должны приводиться к канонической форме;
- необходимость фильтрации ложных срабатываний – внутренние адреса, доверенные домены, известные benign-индикаторы должны помечаться, но не удаляться, чтобы сохранить аудируемость;
- дорогостоящие операции обогащения – GeoIP, reverse DNS, WHOIS являются I/O-bound и лимитированными, что диктует необходимость абстракции и потенциального кэширования.

Общая архитектура. Система построена в соответствии с принципами функциональной конвейерной обработки (*pipeline processing*) и разделения ответственности (*separation of concerns*). Архитектурно выделяются следующие слои:

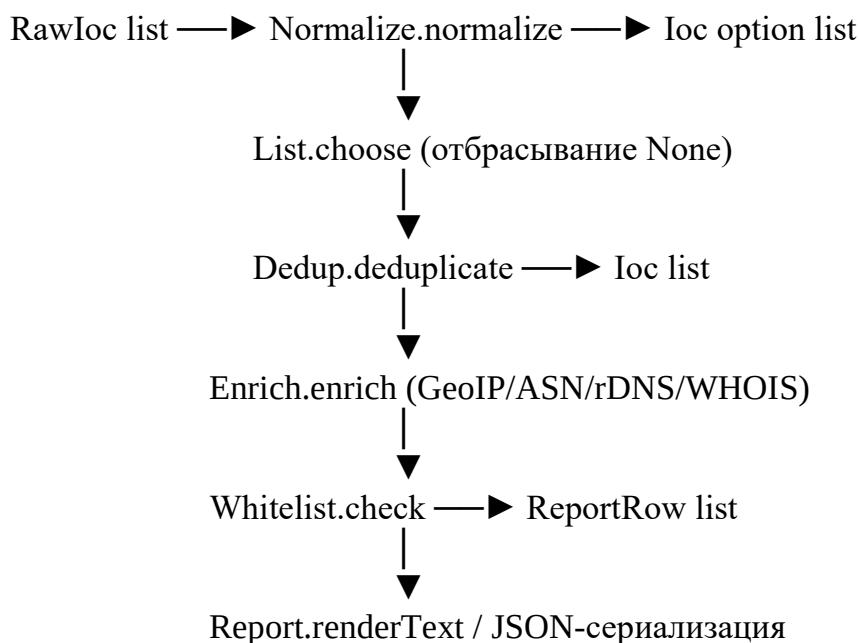
- 1) слой моделей данных – алгебраические типы RawIoc, Ioc, Enrichment, ReportRow;
- 2) слой извлечения (*Extraction*) – модуль Parsing с функциями parseCsv, parseJson, parseStixLite;
- 3) слой нормализации (*Transformation*) – модуль Normalize с функциями canonical, guessKind, normalize;
- 4) слой дедупликации – модуль Dedup.
- 5) слой обогащения – интерфейсы IGeoIp, IAsn, IReverseDns, IWhois и модуль Enrich.

- 6) слой применения правил – тип `WhitelistRule` и модуль `Whitelist`;
7) слой отчётности – модуль `Report`;
8) оркестратор – модуль `Pipeline`, собирающий этапы в единый конвейер.

Такая декомпозиция обеспечивает:

- тестируемость – каждый слой представляет собой чистую функцию или набор чистых функций с явными зависимостями;
- заменяемость – реализации обогащения скрыты за интерфейсами, что позволяет подменять их в тестах и на разных средах;
- композиционность – этапы соединяются оператором конвейера `|>`, что делает поток данных визуально читаемым.

Диаграмма потока данных:



ПОДРОБНОЕ ОПИСАНИЕ КОМПОНЕНТОВ И ОБОСНОВАНИЕ РЕШЕНИЙ

Модель данных. Тип `IocKind`

```
type IocKind = IPv4 | IPv6 | Domain | Url | Md5 | Sha1 |  
Sha256
```

Обоснование. Использование размеченного объединения (*discriminated union*) вместо строкового перечисления обеспечивает:

- исчерпывающее сопоставление с образцом – компилятор предупредит о непокрытых случаях при добавлении нового вида;
- отсутствие недопустимых состояний – невозможно передать произвольную строку вместо типа;
- самодокументируемость – множество допустимых типов зафиксировано в одном месте.

Отдельный случай IPv6 введён, поскольку IPv6-адреса принципиально отличаются от IPv4 и по синтаксису, и по способу канонизации (компактная форма через `IPAddress.ToString()`), и по способу проверки принадлежности подсети (побайтовое сравнение вместо 32-битной маски).

Типы *RawIoc* и *Ioc*. Разделение сырого и нормализованного представления является принципиальным. *RawIoc* содержит значение в исходной форме, имя источника, возможно отсутствующий тип, произвольные теги и метку времени. *Ioc* — канонизированное значение, гарантированно валидный тип, агрегированный список источников и тегов.

Обоснование. Такое разделение реализует принцип «сделать недопустимые состояния невыразимыми» (*make illegal states unrepresentable*). После этапа нормализации тип индикатора известен достоверно, значение канонизировано; все последующие этапы работают с инвариантами, проверенными однократно. Это устраняет необходимость повторных проверок и снижает вероятность ошибок.

Tun Enrichment.

```
type Enrichment = {  
    GeoCountry : string option  
    GeoCity    : string option  
    Asn        : string option  
    AsnOrg     : string option  
    ReverseDns : string option  
    Whois      : string option  
}
```

Обоснование. Поля представлены типом `option`, поскольку обогащение принципиально может не дать результата (IP не найден в MMDB, WHOIS недоступен, DNS-запись отсутствует). Использование `null` в F# не идиоматично и приводит к неявным `NullReferenceException`; тип `option` делает отсутствие значения явным и заставляет клиента обрабатывать этот случай.

Tun ReportRow. Содержит IOC, обогащение, булев флаг ложноположительного срабатывания и список причин отнесения. Сохранение причин в виде строк, а не удаление FP-записей, обеспечивает аудируемость: аналитик может увидеть, почему индикатор был отфильтрован, и при необходимости скорректировать правила. Причин может быть несколько одновременно (например, индикатор попал под правило по суффиксу домена и одновременно имеет метку `benign`), поэтому поле имеет тип `string list`, а не `string option`.

Слой парсинга. Обоснование выбора *FSharp.Data*. Библиотека *FSharp.Data* предоставляет типобезопасные парсеры CSV и JSON с эргономичным API (`CsvFile.Load`, `JsonValue.Load`, `GetColumn`, `TryGetProperty`). Использование проверок на наличие колонки/свойства перед доступом позволяет корректно обрабатывать отсутствующие поля, что критично при работе с гетерогенными фидами.

Функция parseCsv

```
let parseCsv (source: string) (path: string) :  
Result<RawIoc list, ParseError>
```

извлекает колонки indicator, type, tags. Теги разделяются по ; и ,, приводятся к нижнему регистру и обрезаются. Если колонка type отсутствует, тип остаётся None – детекция произойдёт позже, на этапе нормализации.

Доступ к колонкам осуществляется через локальный helper tryGetColumn, построенный на публичном свойстве CsvRow.Columns и методе GetColumn. Причина: в используемой версии FSharp.Data метод TryGetColumn отсутствует, и обращение к нему не компилируется. Helper возвращает string option и семантически полностью эквивалентен TryGetColumn, но не зависит от версии пакета.

Обоснование. Логика допускает частично заполненные строки, что соответствует реальности: фиды часто не гарантируют наличие всех полей. Параметр source явно передаётся, а не выводится из имени файла, поскольку один и тот же файл может быть переименован при передаче.

Возвращаемое значение – Result<RawIoc list, ParseError>, а не просто список: ошибки парсинга фиксируются в структурированном виде (источник, путь, сообщение), не приводя к падению конвейера.

Функция parseJson ожидает массив объектов с полями value, опционально type, tags.

Обоснование. Схема минимальна и покрывает наиболее распространённый случай; расширение возможно без нарушения существующего кода. Возврат Result<_, ParseError> – по тем же причинам, что и в parseCsv.

Функция parseStixLite обрабатывает подмножество STIX 2.x: объекты с полем pattern вида [ipv4-addr:value = '1.2.3.4'] и массивом labels.

Обоснование. Полноценный STIX 2.x чрезвычайно объёмен (десятки типов объектов, сложная графовая модель). Для задач агрегации ИОС достаточно извлечь значение и тип из pattern. Регулярное выражение =\s*('[^']+) извлекает значение между кавычками; тип определяется по подстроке (ipv4-addr, ipv6-addr, domain-name, url, file:hashes). Такой подход уязвим к экзотическим паттернам, но для типовых фидов достаточен и легко расширяется. Из одного pattern извлекается одна пара «путь-значение» – этого достаточно для типовых однозначных паттернов; для паттернов с несколькими индикаторами реализация вернёт только первый.

Слой нормализации. Регулярные выражения:

```
let reMd5      = Regex(@"^[a-f0-9]{32}$",  
RegexOptions.IgnoreCase)  
let reSha1     = Regex(@"^[a-f0-9]{40}$",  
RegexOptions.IgnoreCase)
```

```
let reSha256 = Regex(@"^[a-f0-9]{64}$",
RegexOptions.IgnoreCase)
let reIpv4 =
Regex(@"^(\d{1,3})\.(\d{1,3})\.(\d{1,3})\.(\d{1,3})$")
let reDomain = Regex(@"^(?=.{1,253}$) ([a-z0-9] (?:[a-z0-9-
]{0,61}[a-z0-9])?\.)+[a-z0-9] (?:[a-z0-9-]{0,61}[a-z0-
9])?$", RegexOptions.IgnoreCase)
```

Обоснование. Регулярные выражения компилируются однократно (на уровне модуля) и переиспользуются – это существенно для производительности. Регулярное выражение для домена использует lookahead (`?=.{1,253}$`) для проверки общей длины (RFC 1035) и посимвольную структуру меток (RFC 1123): метка не может начинаться или заканчиваться дефисом, длина метки не превышает 63 символов. Домен верхнего уровня не ограничен алфавитным шаблоном `[a-z]{2,}`, поскольку в современных условиях существуют как punycode-домены (`xn--...`), так и числовые TLD; ограничение на длину и структуру меток уже отсеивает заведомо некорректные значения.

Функция `defang`:

```
let private defang (s: string) =
    s.Replace("[.]", ".").Replace("(.)",
".").Replace("{.}", ".")
    .Replace("[dot]", ".")
    .Replace("[:]", ":")
    .Replace("hxxp", "http")
    .Replace("hXXp", "http")
```

Обоснование. Дефангирование – распространённая практика в OSINT-отчётах, при которой потенциально опасные индикаторы намеренно искажаются, чтобы предотвратить случайное срабатывание. Обратная операция необходима для канонизации. Порядок замен фиксирован: сначала точечные формы `[.]`, `(.)`, `{.}`, `[dot]`, затем `[:]`, затем `hxxp/hXXp`. Замена `hxxp` на `http` производится дважды для регистров `hxxp` и `hXXp` – в реальных данных встречаются оба варианта. Все замены выполняются с флагом `RegexOptions.IgnoreCase` и `CultureInfo.InvariantCulture`, что делает поведение независимым от текущей локали.

Область применения `hxxp`-замены ограничена контекстом схемы: замена срабатывает только тогда, когда за `hxxp` или `hxxps` непосредственно следует `://`. Это предотвращает ложные срабатывания внутри посторонних токенов (например, `myhxxp`, `hxxp.example.com`, `path/hxxp`), но корректно обрабатывает дефангированные URL вида `hxxp[:]//example.com` (так как замена `[:]` на `:` выполняется раньше).

Функция `guessKind` реализует каскадную эвристику: сначала проверяется строковое имя типа (`"ipv4"`, `"ipv6"`, `"domain"`, `"url"`, `"md5"`, `"sha1"`, `"sha256"` и их

варианты с дефисом), затем – само значение по последовательности регулярных выражений (хэши → IPv4 → IPv6 → URL → домен).

Обоснование. Порядок проверок неслучаен: хэши проверяются первыми, поскольку строковое представление хэша однозначно; IP предшествует домену, так как IP-адрес мог бы быть ошибочно распознан как домен (например, 1.2.3.4 формально подходит под некоторые слабые шаблоны домена). Проверка URL предшествует домену, поскольку URL содержит домен в качестве подстроки. IPv6 проверяется после IPv4, чтобы IPv4-mapped-адреса вида ::ffff:1.2.3.4 детектировались как IPv4 по первой совпавшей ветке каскада.

Функция canonical

```
let canonical (raw: string) (kind: IocKind option) :  
(string * IocKind) option
```

выполняет три операции: дефангирование, определение типа (переданный или угаданный), валидацию и канонизацию значения по типу.

Обоснование. Для IPv4 выполняется не только синтаксическая, но и семантическая проверка – каждый октет должен принадлежать диапазону [0, 255]. Это отсеивает ложные срабатывания вида 999.999.999.999, которые могут проходить регулярное выражение. Каноническая форма IPv4 нормализует ведущие нули (01.02.03.04 → 1.2.3.4). Для IPv6 канонизация выполняется через IPAddress.ToString() с приведением к нижнему регистру; квадратные скобки, если присутствовали, отбрасываются. Для домена выполняется удаление завершающей точки (FQDN-нотация example.com. эквивалентна example.com) и приведение к нижнему регистру, поскольку DNS-имена регистронезависимы (RFC 4343). Для URL схема и хост приводятся к нижнему регистру, а исходный регистр пути, запроса и фрагмента сохраняется – пути в URL регистрозависимы. Дефолтные порты (80 для http, 443 для https, 21 для ftp) опускаются; пустой путь нормализуется в /.

Возврат option сигнализирует о невозможности канонизации – такие записи отбрасываются на этапе List.choose.

Слой дедупликации. Ключ дедупликации:

```
let private key (i: Ioc) = (i.Kind, i.Value)
```

Обоснование. Пара (Kind, Value) является естественным ключом: одно и то же значение может теоретически встречаться в разных типах (например, строка из 32 hex-символов может быть как MD5-хэшем, так и – теоретически – частью домена), поэтому тип должен входить в ключ. Использование кортежа в качестве ключа словаря корректно, поскольку F# реализует структурное сравнение и хеширование для кортежей.

Операция слияния:

```
let private merge (a: Ioc) (b: Ioc) = {  
  a with
```

```

        Sources = (a.Sources @ b.Sources) |>
List.distinct
        Tags     = (a.Tags @ b.Tags) |> List.distinct
        SeenAt   = max a.SeenAt b.SeenAt
    }

```

Обоснование. При дублировании индикатора сохраняется объединение метаданных: список источников и тегов пополняется, время наблюдения берётся как максимум (последнее наблюдение). Это соответствует семантике агрегации разведданных: индикатор, встречающийся в нескольких независимых фидах, считается более достоверным, и эта информация не должна теряться. Результат слияния сортируется (`List.sort`) для детерминированности вывода.

Алгоритм. Используется `Dictionary<_, Ioc>` с императивным свёртыванием через `Seq.fold`. Сложность – $O(N)$ по количеству индикаторов.

Обоснование. Альтернативный функциональный подход (`List.groupBy + List.map`) требует двух проходов и создания промежуточных коллекций. Императивное свёртывание с хеш-таблицей эффективнее по памяти и времени, что критично при обработке миллионов индикаторов. `List.distinct` внутри слияния может иметь сложность $O(L^2)$ по длине списка, но поскольку списки источников и тегов обычно короткие (единицы элементов), это не является узким местом.

Слой обогащения. Интерфейсы:

```

type IGeoIp      = abstract Lookup : string -> (string *
string) option
type IAsn        = abstract Lookup : string -> (string *
string) option
type IReverseDns = abstract Lookup : string -> string
option
type IWhois      = abstract Lookup : string -> string
option

```

Обоснование. Каждый источник обогащения абстрагирован отдельным интерфейсом с единственным методом. Это решение обосновано несколькими соображениями:

- 1) принцип разделения интерфейсов (ISP) – клиент, нуждающийся только в GeoIP, не зависит от WHOIS;
- 2) тестируемость – в unit-тестах подставляются детерминированные заглушки без сетевых вызовов;
- 3) заменяемость реализаций – продакшн-реализация (`MaxMind.Db`, `Dns.GetHostEntry`, whois-клиент с кэшем) и тестовая (`in-memory Map`) взаимозаменяемы;

4) возврат option – явно сигнализирует о возможности отсутствия данных, что соответствует реальности (не каждый IP есть в MMDB, не у каждого IP есть PTR-запись).

«Заглушки». Реализации на основе Map<string, _>. Параметр конструктора – неизменяемое отображение, что обеспечивает потокобезопасность без блокировок.

Обоснование. «Заглушки» присутствуют в самом демонстрационном файле, чтобы код был самодостаточен и запускаем без внешних сервисов. В продакшне они заменяются реальными реализациями, интерфейс при этом не меняется.

Функция enrich:

```
let enrich (geo: IGeoIp) (asn: IAsn) (rdns: IReverseDns)
(whois: IWhois) (ioc: Ioc) : Enrichment
```

Диспетчеризация по типу индикатора:

- IPv4, IPv6 → GeoIP + ASN + reverse DNS;
- Domain → WHOIS;
- Url → извлечение хоста, рекурсивное обогащение по типу хоста;
- хэши → пустое обогащение.

Обоснование. Хэши не поддаются сетевому обогащению в классическом смысле (для них нужны сервисы вроде VirusTotal, что выходит за рамки задачи). Для URL извлечение хоста производится через System.Uri, при этом исключение перехватывается и возвращается None – некорректные URL не должны приводить к падению конвейера. Рекурсивное использование enrichIp/enrichDomain для URL устраняет дублирование логики.

Слой whitelist-правил. Тип WhitelistRule:

```
type WhitelistRule =
| ExactValue    of IocKind * string
| DomainSuffix  of string
| CidrRange     of string * int
| TagMatch      of string
```

Обоснование. Размеченное объединение покрывает четыре наиболее распространённых класса правил:

- ExactValue – точное совпадение по паре (тип, значение) для внесения конкретных индикаторов в белый список;
- DomainSuffix – совпадение по суффиксу для покрытия целых доменных зон (example.com покрывает www.example.com, api.example.com);
- CidrRange – совпадение по подсети для покрытия внутренних диапазонов (RFC 1918) и доверенных сетей; поддерживаются как IPv4 (32-битная маска), так и IPv6 (побайтовое сравнение);

- TagMatch – фильтрация по метке, позволяющая пометить benign-индикаторы декларативно.

Расширение множества правил (например, добавление регулярных выражений) возможно без нарушения существующего кода – компилятор укажет на неисчерпывающие сопоставления.

Функция check

```
let check (rules: WhitelistRule list) (ioc: Ioc) : string
list
```

возвращает список сработавших правил (пустой список – если ни одно правило не сработало).

Обоснование. Возврат причин (а не просто bool) обеспечивает аудируемость: аналитик видит, какие именно правила сработали. Это критично при отладке whitelist-логики и при разборе инцидентов, когда индикатор был ошибочно помечен как FP. Индикатор может одновременно попадать под несколько правил (например, и по суффиксу домена, и по тегу); возврат списка сохраняет всю эту информацию, тогда как string option зафиксировал бы только первое сработавшее правило.

Для URL правила проверяются как на сам URL (через ExactValue и TagMatch), так и на извлечённый хост (через DomainSuffix и CidrRange). Это соответствует ожиданиям: правило DomainSuffix "example.com" должно срабатывать и на Ioc типа Url со значением https://api.example.com/x.

Реализация CIDR-проверки. IPv4:

```
let private ipToUInt32 (ip: string) =
    let parts = ip.Split('.') |> Array.map UInt32.Parse
    (parts.[0] <<< 24) ||| (parts.[1] <<< 16) |||
    (parts.[2] <<< 8) ||| parts.[3]
```

```
let private inIpv4Cidr (ip: string) (network: string)
(prefix: int) =
    if prefix < 0 || prefix > 32 then false
    elif not (Kinds.isIpv4 ip && Kinds.isIpv4 network)
then false
    else
        let ipN = ipToUInt32 ip
        let netN = ipToUInt32 network
        let mask = if prefix = 0 then 0u else
UInt32.MaxValue <<< (32 - prefix)
        (ipN &&& mask) = (netN &&& mask)
```

IPv6:

```

let private inIpv6Cidr (ip: string) (network: string)
(prefix: int) =
    if prefix < 0 || prefix > 128 then false
    else
        match ipv6Bytes ip, ipv6Bytes network with
        | Some ipBytes, Some netBytes ->
            let fullBytes = prefix / 8
            let remainingBits = prefix % 8
            let fullEqual =
                [0 .. fullBytes - 1]
                |> List.forall (fun i -> ipBytes.[i] =
netBytes.[i])
            if not fullEqual then false
            elif remainingBits = 0 then true
            else
                let mask = byte (0xFF <<< (8 -
remainingBits))
                (ipBytes.[fullBytes] &&& mask) =
(netBytes.[fullBytes] &&& mask)
            | _ -> false

```

Обоснование. IPv4-адрес преобразуется в 32-битное беззнаковое целое, маска вычисляется сдвигом. Операция &&& (побитовое И) с маской выделяет сетевую часть. Специальный случай `prefix = 0` обрабатывается отдельно, поскольку сдвиг на 32 бита для 32-битного типа в .NET даёт неопределённое поведение (сдвиг по модулю 32). Для IPv6 побайтовое сравнение выбрано как более простое и не зависящее от платформенных особенностей работы с 128-битными целыми. Некорректные IP отсеиваются проверкой `Kinds.isIpv4` (для IPv4) или `IPAddress.TryParse` (для IPv6) – конвейер не должен падать из-за некорректного правила.

Слой отчётности. Функция `toRows`

```

let toRows (enrich: Ioc -> Enrichment) (rules:
WhitelistRule list) (iocs: Ioc list) : ReportRow list

```

принимает функцию обогащения, а не сам конвейер обогащения. Это позволяет вызывающей стороне передать частично применённую `Enrich.enrich` с зафиксированными зависимостями.

Обоснование. Такой приём (передача функции вместо зависимости) увеличивает гибкость: в тестах можно передать простую лямбду, возвращающую фиктивное обогащение, без создания реализаций всех четырёх интерфейсов.

Функция `renderText` формирует человекочитаемое представление через `StringBuilder`.

Обоснование. StringBuilder вместо конкатенации строк через + или String.concat – стандартная оптимизация для построения длинных строк (избегает $O(N^2)$ аллокаций). Формат вывода включает как сами индикаторы, так и метаданные (источники, теги, обогащение, причины FP), что делает отчёт самодостаточным для передачи аналитику. Отсутствующие опциональные поля обогащения выводятся как -.

Функция toJson формирует сериализуемое представление отчёта через JsonValue.Record.

Обоснование. Помимо человекочитаемого текста, отчёт должен быть пригоден для машинной обработки и передачи в SIEM/SOAR-системы. Формат JSON выбран как де-факто стандарт обмена структурированными данными. Поле whitelist_reasons представлено массивом строк (в соответствии с тем, что индикатор может попадать под несколько правил одновременно). Отсутствующие опциональные поля представляются как JSON-null, а не опускаются, что упрощает клиентам разбор фиксированной схемы.

Оркестратор:

```
let runWithDiagnostics (cfg: PipelineConfig) (rawFeeds:
RawIoc list) : ReportRow list * RunDiagnostics =
    let enrich = Enrich.enrich cfg.GeoIp cfg.Asn
    cfg.ReverseDns cfg.Whois
    let normalized = rawFeeds |> List.choose
Normalize.normalize
    let dropped = rawFeeds.Length - normalized.Length
    let deduped = normalized |> Dedup.deduplicate
    let duplicatesRemoved = normalized.Length -
deduped.Length
    let rows = deduped |> Report.toRows enrich
    cfg.Whitelist
    rows, {
        InputCount = rawFeeds.Length
        NormalizedCount = normalized.Length
        DroppedCount = dropped
        DuplicatesRemoved = duplicatesRemoved
    }

let run (cfg: PipelineConfig) (rawFeeds: RawIoc list) :
ReportRow list =
    fst (runWithDiagnostics cfg rawFeeds)
```

Обоснование. Оркестратор представлен функцией, читаемой как декларативное описание потока данных. Каждый этап – независимая функция с чётко определённой сигнатурой. Конфигурация (PipelineConfig) инкапсулирует все зависимости, что упрощает передачу конвейера между слоями приложения и его параметризацию.

Возврат диагностики (RunDiagnostics) вместе с результатами позволяет наблюдать поведение конвейера: сколько записей было отброшено на нормализации, сколько дубликатов удалено. Это существенно при отладке нештатных ситуаций, когда итоговый отчёт не совпадает с ожидаемым. Поле DuplicatesRemoved хранит именно число удалённых дубликатов, а не число записей после дедупликации – именование соответствует семантике.

Порядок этапов принципиален:

- 1) нормализация предшествует дедупликации, поскольку дедупликация по каноническим ключам требует нормализованных значений;
- 2) дедупликация предшествует обогащению, поскольку обогащение дорогостояще – нет смысла обогащать дубликаты;
- 3) whitelist-проверка выполняется после обогащения, поскольку некоторые правила могли бы использовать данные обогащения (в текущей реализации – нет, но архитектура это допускает).

Принятые проектные решения и их обоснование. Функциональный стиль против объектного. Выбор функционального стиля (алгебраические типы, чистые функции, композиция через $|>$) обусловлен природой задачи: конвейерная обработка данных естественно выражается через композицию преобразований. Имутабельность моделей исключает целый класс ошибок, связанных с разделяемым изменяемым состоянием. Сопоставление с образцом по размеченным объединениям обеспечивает исчерпывающую обработку случаев на этапе компиляции.

Единственное место с императивным стилем – дедупликация через Dictionary, что является осознанной оптимизацией по производительности и памяти.

Использование option вместо исключений. Все операции, которые могут не дать результата (нормализация, обогащение), возвращают option. Для операций парсинга, где необходимо передать диагностическую информацию, используется Result<_, ParseError>. Исключения используются только для настоящего исключительных ситуаций (некорректный IP внутри CIDR-правила – перехватывается локально).

Обоснование. Исключения в F# считаются антипаттерном для ожидаемых ситуаций: они неявны в сигнатуре, дороги по производительности и усложняют композицию. option и Result делают возможность отсутствия значения или ошибки частью типа и заставляют клиента обрабатывать этот случай.

Абстракция обогащения через интерфейсы. Альтернативой интерфейсам могли бы быть:

- функции высшего порядка – string -> (string * string) option вместо IGeoIp;
- записи с полями-функциями;
- DU с реализациями.

Выбраны интерфейсы по следующим причинам:

- явность – интерфейс имеет имя и документирует контракт;
- совместимость с .NET – реализации могут быть написаны на C#;

- возможность DI-контейнеров – стандартные механизмы внедрения зависимостей в .NET работают с интерфейсами.

Возврат причин FP в виде списка строк. Альтернативой строке был бы размеченный тип `WhitelistMatch` с полями для каждого вида правила. Строка выбрана как компромисс между простотой и информативностью: причина предназначена для человеческого чтения в отчёте, а не для машинной обработки. Список (а не `option`) необходим потому, что индикатор может одновременно удовлетворять нескольким правилам; потеря этой информации снизила бы аудируемость. При необходимости тип можно заменить без изменения сигнатуры `check` (тип возврата – `string list`).

Использование *FSharp.Data* для парсинга. Альтернативы: `System.Text.Json`, `Newtonsoft.Json`, `CsvHelper`, ручные парсеры. `FSharp.Data` выбран благодаря идиоматичному для F# API: `JsonValue` позволяет работать с JSON без предварительного объявления типов (важно при гетерогенных схемах), `CsvFile` обеспечивает типобезопасный доступ к колонкам с обработкой отсутствующих значений.

Отказ от удаления FP-записей. Ложноположительные индикаторы не удаляются из отчёта, а помечаются. Это решение принято по следующим причинам:

- аудируемость – можно увидеть, что индикатор был в фиде, но отфильтрован;
- анализ качества правил – статистика по FP позволяет выявлять слишком широкие правила;
- гибкость потребителя – потребитель отчёта сам решает, игнорировать FP или нет.

Канонизация URL с сохранением регистра пути. URL нормализуется по частям: схема и хост приводятся к нижнему регистру, регистр пути, запроса и фрагмента сохраняется. Это соответствует RFC 3986: схема и хост регистронезависимы, путь – регистрозависим.

Обоснование. В более ранних версиях канонизации URL приводила к нижнему регистру всю строку целиком, что приводило к ложному слиянию различных ресурсов (`/Path` и `/path` – разные пути на большинстве серверов). Сохранение регистра пути корректно с точки зрения стандарта; при этом дефолтные порты и пустой путь всё равно нормализуются, что устраняет наиболее частые варианты дублирования.

Разделение CSV-доступа через *helper*. Прямое использование `row.TryGetColumn` заменено на локальный `helper tryGetColumn`, построенный на `row.Columns` и `row.GetColumn`. Причина прикладная: в используемой версии `FSharp.Data` метод `TryGetColumn` отсутствует, и обращение к нему не компилируется. `Helper` семантически эквивалентен и не зависит от версии пакета.

Ограничение области *hxxp*-замены. Замена `hxxp/hxxps` на `http/https` выполняется только в позиции схемы – с использованием `lookahead (?=://)`. Это устраняет ложные срабатывания при обработке токенов, содержащих подстроку `hxxp` (например, `myhxxp://...` или `hxxp.example.com`).

Дефангированные URL вида `hxxp[:]//example.com` при этом по-прежнему обрабатываются корректно, поскольку замена `[:]` на `:` выполняется на предыдущем шаге.

Направления развития:

- 1) параллелизация обогащения – `Array.Parallel.map` или `Async.Parallel` для I/O-bound операций;
- 2) кэш с TTL – `ConcurrentDictionary` или `MemoryCache` для дедупликации дорогостоящих запросов;
- 3) реальные реализации обогащения – `MaxMind.Db.Reader` для MMDB, `Dns.GetHostEntryAsync` для rDNS, whois-клиент с rate limiting;
- 4) стриминговая обработка – замена `list` на `seq` или `IAsyncEnumerable` для работы с фидами, не помещающимися в память;
- 5) персистентность – `SQLite/DuckDB` для хранения истории индикаторов и их обогащения;
- 6) property-based тестирование – `FsCheck` для проверки инвариантов нормализации (идемпотентность, сохранение семантики при дефангировании);
- 7) метрики – счётчики по типам индикаторов, источникам, FP-срабатываниям для мониторинга.

Заключение. Представленная реализация демонстрирует применение принципов функционального программирования к задаче обработки OSINT-данных. Ключевыми архитектурными решениями являются: разделение сырых и нормализованных моделей, конвейерная композиция этапов, абстракция внешних источников обогащения через интерфейсы, применение размеченных объединений для whitelist-правил и сохранение причин FP-срабатываний для аудируемости. Система расширяема: добавление новых форматов фидов, источников обогащения и типов правил не требует изменения существующих компонентов, а лишь добавления новых реализаций или ветвей сопоставления. Ограничения текущей реализации (однопоточность, отсутствие кэша и персистентности) осознанны и очерчивают направления дальнейшего развития.